# WordleBot on AWS

## Video Presentation Script

Deploying a full-stack web application on a single Amazon EC2 instance - securely and at near-zero cost

| | |
|---|---|
| Presenters | Diya Bhagavati Alla and Shubhashree D |
| Course | AWS and Cloud Computing |
| Length | About 10 minutes |
| Region | ap-south-1 (Mumbai) |
| Stack | React + Vite, Python FastAPI, nginx, systemd, Ubuntu 24.04 |

# 1. Before you record

• Start the instance: `aws ec2 start-instances --instance-ids <id> --region ap-south-1`

• Note the new public IP (it changes on every start) and update the port-22 rule to your current IP.

• Wait about 10 seconds after boot for the backend to load its precomputed patterns, then open the app once to confirm it works.

• Open these tabs: EC2 Instances, Security Groups, Key Pairs, Cost Explorer, a terminal SSHed into the instance, and the app.

• Hide or blur your AWS account ID, key file contents and home IP address when they appear on screen.

# 2. Architecture (show as a diagram)

```
Browser --HTTP :80--> [ EC2 t3.micro: nginx ] --/api/*--> [ FastAPI (uvicorn) on 127.0.0.1:8000 ]
nginx also serves the built React app from /home/ubuntu/app/frontend-web/dist
```

| AWS resource | Configuration | Why |
|---|---|---|
| EC2 instance | t3.micro, Ubuntu 24.04 AMI | Smallest burstable type that runs the app; free-tier eligible |
| EBS volume | 10 GB gp3, delete on termination | Cheaper and faster than gp2; no orphaned volumes |
| Security group | 80 from 0.0.0.0/0; 22 from my IP /32 only | Public website, private admin access |
| Key pair | wordlebot-key (.pem kept locally) | Password-less, key-based SSH |
| Network | Default VPC + default public subnet | No extra resources or cost |
| Public IP | Auto-assigned (not Elastic IP) | Free while running; changes on restart |

# 3. Scene-by-scene script

## 0:00 - Introduction

**On screen:** Title slide, then the app home page

**Say:** Hi, we're Diya Bhagavati Alla and Shubhashree D. In this video we'll walk you through how we deployed WordleBot - a full-stack Wordle solver - on Amazon EC2. I'll cover the architecture, how I launched and secured the instance, how the app runs on the server, and how I kept the AWS bill close to zero. The constraint I set myself was simple: use only one EC2 instance and nothing else.

## 0:40 - What the app does

**On screen:** Play one round in the browser

**Say:** WordleBot helps you solve Wordle. You type the guess you made, set each tile to green, yellow or grey to match the game, and the bot suggests the best next word. The frontend is React built with Vite, and the backend is a Python FastAPI service that runs the solving algorithm.

## 1:20 - Architecture

**On screen:** Architecture diagram

**Say:** Everything runs on a single instance. Nginx listens on port 80. It serves the built frontend files directly, and forwards any request starting with /api to the FastAPI backend. The backend listens only on 127.0.0.1, port 8000, so it can't be reached from the internet at all - only through nginx. That gives us one public entry point and a smaller attack surface.

## 2:10 - Choosing the instance

**On screen:** EC2 console: instance details tab

**Say:** I chose a t3.micro in the Mumbai region, which is closest to me for lower latency. It has 2 vCPUs and 1 GB of memory. T3 is a burstable family: it earns CPU credits while idle and spends them under load, which suits an app with short bursts of work. For the operating system I used the official Ubuntu 24.04 AMI, and for storage a 10 GB gp3 volume, which costs less than gp2 and is set to delete when the instance is terminated.

```
aws ec2 run-instances --image-id <ubuntu-24.04-ami> --instance-type t3.micro \
  --key-name wordlebot-key --security-group-ids <sg-id> \
  --block-device-mappings DeviceName=/dev/sda1,Ebs={VolumeSize=10,VolumeType=gp3}
```

## 3:00 - Networking and security

**On screen:** Security group inbound rules

**Say:** I used the default VPC and its public subnet, so I didn't need to create any networking. Security is handled by one security group acting as a virtual firewall. Port 80 is open to everyone because the website is public. Port 22, for SSH, is open only to my own IP address as a /32. This is the principle of least privilege. Security groups are stateful, so I don't need outbound rules for responses. When my home IP changed, I replaced the SSH rule with the new address.

```
aws ec2 authorize-security-group-ingress --group-id <sg-id> --protocol tcp --port 22 --cidr <my-ip>/32
aws ec2 authorize-security-group-ingress --group-id <sg-id> --protocol tcp --port 80 --cidr 0.0.0.0/0
```

## 3:50 - Access and deployment

**On screen:** Terminal: scp, then ssh

**Say:** To log in, I created an EC2 key pair. AWS keeps the public key and I keep the private .pem file, so there are no passwords. I packaged the backend and frontend code into a small tar file, copied it with scp, and SSHed in. On the server I installed Python, Node.js and nginx, created a Python virtual environment for the backend, and built the frontend there with npm. I deliberately avoided Docker and extra AWS services to keep it simple and cheap.

```
scp -i wordlebot-key.pem app.tar.gz ubuntu@<public-ip>:~
ssh -i wordlebot-key.pem ubuntu@<public-ip>
cd app/frontend-web && npm install && npm run build
```

## 4:50 - Running it reliably

**On screen:** Terminal: systemctl status; nginx site config

**Say:** The backend runs as a systemd service. That means it starts automatically when the instance boots and restarts itself if it crashes. Nginx is enabled the same way. So if the instance is stopped and started again, the app comes back on its own with no manual steps. One problem I hit: nginx returned 500 errors because Ubuntu's home directory permissions stopped it from reading the frontend files. Checking the nginx error log pointed straight to it, and adding execute permission on the directory fixed it.

```
sudo systemctl status wordlebot-backend nginx
location /api/ { proxy_pass http://127.0.0.1:8000/; }
```

## 5:50 - Live demo

**On screen:** Browser on http://<public-ip>

**Say:** Here's the live app on the instance's public IP. I'll enter the bot's opening word, SALET, set the colours from my game, and ask for the next guess. The answers come back in well under a second. Behind the scenes, the backend precomputes the colour pattern for every guess-and-answer pair when it starts, then searches ahead to choose the guess that solves the puzzle fastest. Tested against all 2,309 official answers, it averages 3.42 guesses and never fails - all on a t3.micro.

## 6:50 - Updating the app

**On screen:** Terminal: scp one file, restart service

**Say:** Updates are straightforward. When I improved the solver, I started the instance, copied the changed file with scp, restarted the systemd service, and tested the API through the public IP before stopping the instance again.

```
scp -i wordlebot-key.pem solver.py ubuntu@<ip>:~/app/backend/services/
ssh ... 'sudo systemctl restart wordlebot-backend'
```

## 7:30 - Cost management

**On screen:** EC2 console: Stop instance; Cost Explorer

**Say:** Cost was a key goal. When I'm not using the app, I stop the instance rather than terminate it. A stopped instance has no compute charge - I only pay for the 10 GB disk, about 8 cents a month - and everything is kept for next time. Terminating would delete it completely. A t3.micro costs about one cent an hour while running. Cost Explorer shows that my total spend for this project is effectively zero.

```
aws ec2 stop-instances --instance-ids <id> --region ap-south-1
aws ce get-cost-and-usage --time-period Start=...,End=... --granularity DAILY --metrics UnblendedCost
```

## 8:30 - Challenges and lessons

**On screen:** Slide with three bullets

**Say:** Three lessons stood out. One: the public IP changes every time the instance starts, so links break; an Elastic IP or a domain name would fix that. Two: some phone browsers try HTTPS first, and because port 443 is closed they just hang; the proper fix is a domain with a free TLS certificate. Three: on a small instance, do heavy work once at startup instead of on every request - that took my slowest response from 21 seconds to under one.

## 9:20 - Future improvements

**On screen:** Slide

**Say:** Next steps would be: HTTPS with a domain and certificate, an Elastic IP, CloudWatch alarms for CPU and a health check, automating the whole setup with a user-data script or infrastructure as code such as CloudFormation, and storing the code in S3 or a Git repository for repeatable deployments.

## 9:50 - Closing

**On screen:** Closing slide

**Say:** So that's WordleBot on AWS: one EC2 instance, a locked-down security group, systemd and nginx keeping it running, and a bill of practically nothing. Thanks for watching.

# 4. AWS concepts to highlight

| Concept | How this project shows it |
|---|---|
| Instance types / burstable CPU | t3.micro chosen for cost; heavy work moved to startup to save CPU credits |
| AMI | Official Ubuntu 24.04 image from Canonical |
| EBS | 10 GB gp3 root volume, delete-on-termination |
| VPC and subnets | Default VPC with a public subnet and auto-assigned public IP |
| Security groups | Stateful firewall; least-privilege SSH; public HTTP only |
| Key pairs | Key-based SSH, private key never leaves the laptop |
| Elastic IP vs public IP | Public IP changes on stop/start - a known trade-off |
| Instance lifecycle | Stop (keep disk, pay storage) vs terminate (delete everything) |
| Cost management | Cost Explorer shows effectively $0; stop when idle |
| AWS CLI | Every step scripted from the terminal |

# 5. Likely questions and answers

**Q: Why not Elastic Beanstalk, Lightsail or containers?**
A: The goal was to learn core EC2 by hand and keep cost minimal with exactly one resource. Those services add convenience but also abstraction and, sometimes, cost.

**Q: Why not use an Elastic IP?**
A: An Elastic IP is free only while attached to a running instance. Since I stop the instance most of the time, I accepted a changing public IP to keep the cost at zero.

**Q: Is the backend secure?**
A: It binds to localhost only, so it is reachable just through nginx. SSH is limited to one IP and uses a key pair. The next step would be HTTPS.

**Q: How would you scale it?**
A: Put instances in an Auto Scaling group behind an Application Load Balancer, created from an AMI or a user-data script. The backend is stateless, so it scales horizontally.

**Q: What happens if the instance reboots?**
A: systemd starts nginx and the backend automatically, so the app recovers without manual steps.